ChatGPT

SilicaFlux Codebase Research Report

Executive summary

SilicaFlux is best understood not as a conventional software application but as an **RTL hardware-design portfolio centred on a custom dataflow-oriented compute architecture**. The supplied repository snapshot contains a coherent primary SilicaFlux implementation alongside a much broader collection of related hardware experiments: RISC-V CPUs, finite-state-machine libraries, DSP blocks, cache/memory designs, a UVM-inspired verification environment, formal-verification examples, an SoC integration project, an ML-controlled hardware optimiser and a Bitcoin SHA-256 ASIC. The supplied PDF independently characterises the broader collection as roughly 30,000 lines of SystemVerilog across 179 RTL/test files and 176 modules, with substantial parameterisation but comparatively weak verification infrastructure. 🗄filecite🗄turn0file0🗄

The **main architectural core** is the nested `silicaflux/` project contained in `silicaflux.zip`. It is much smaller and more internally consistent than the full archive: 16 RTL files totalling about 3,440 lines, seven SystemVerilog testbenches totalling about 1,142 lines, a Python assembler, synthesis scripts for AMD/Xilinx Vivado and Intel/Altera Quartus, an SDC timing file and an ISA reference. Its architecture combines an in-order scalar pipeline with a scheduler-driven accelerator fabric comprising four compute tiles, SIMD vector execution, a sequential 4×4 tensor unit, per-tile scratchpad memory, a 2×2 XY-routed NoC and a serialised chiplet boundary interface. This is a credible teaching/prototyping architecture, but the present implementation should **not yet be treated as production semiconductor IP**.

The strongest aspects are architectural decomposition, use of SystemVerilog packages and packed structs, explicit ready/valid protocols, parameterisation, separate simulation and FPGA synthesis flows, and a thoughtfully documented custom ISA. These are materially better engineering practices than a collection of unrelated HDL examples. The supplied prior audit similarly found strong parameterisation and modularity across the wider repository. 🗄filecite🗄turn0file0🗄

The main weaknesses are more consequential. The repository has **no visible licence for SilicaFlux itself**, so redistribution, commercial reuse and contribution rights are unclear. The canonical source is buried inside ZIP archives rather than represented as ordinary version-controlled directories. No GitHub Actions, GitLab CI or Bitbucket Pipelines configuration is present in the supplied snapshot. The main core contains almost no SystemVerilog assertions, no functional coverage, no reference-model checking and no formal properties. Several tests print success or increment pass counters without actually checking architectural results. The run script equates a lack of lines beginning `[FAIL]` with success, which makes false-positive regressions possible.

There are also concrete RTL risks. The chiplet receive path samples a multi-bit asynchronous data bus through independent two-flop registers; that is not a safe general-purpose CDC scheme for a multi-bit bus. The advertised parity mechanism is only a placeholder and is not transmitted as a separate parity bit. Scheduler write-back loses destination-register identity and suppresses zero-valued tile results. Some architectural constants are nominally parameterised yet logic is hard-coded for exactly four tiles. The tensor unit's real documented latency in source is about 66 cycles, whereas the top-level README

says roughly 20 cycles, revealing specification drift. Those issues should be addressed before pursuing FPGA benchmarking, ASIC synthesis or external IP reuse.

A public-hosting search performed on 2 November 2026 did **not surface an exact public repository corresponding to the supplied SilicaFlux snapshot**. Searches for `"SilicaFlux" GitHub`, `"Silica Flux" SystemVerilog GitHub`, `site:gitlab.com SilicaFlux SystemVerilog`, `site:bitbucket.org SilicaFlux`, the distinctive README title `"A Modular Dataflow-Oriented Semiconductor Compute Architecture"`, `"SilicaFlux v1.0" semiconductor architecture RTL`, `"SilicaFlux" "Dataflow Scheduler" "DFSYNC"`, and `"silicaflux_top" "tensor_unit" SystemVerilog` produced unrelated results such as SiliconFlow and ByteDance Flux rather than this codebase. [1] I therefore treat the user-supplied archive as the authoritative source snapshot rather than inferring repository stars, commit activity or issue history that cannot be verified.

Repository identity, purpose and architecture

Public-repository search

The searches requested in the brief produced no credible exact match. In particular, the exact architecture name, distinctive module names and `DFSYNC` terminology were not discoverable in indexed GitHub/GitLab/Bitbucket results. Search results instead resolve to projects with similar words, such as SiliconFlow and ByteDance's unrelated GPU communication library called Flux. [1] This means there is no defensible basis for reporting stars, forks, recent commits or issue activity for SilicaFlux itself.

The supplied snapshot also contains no `.git` metadata, so commit dates and authorship history cannot be reconstructed locally. File timestamps in the outer ZIP reflect the supplied snapshot rather than meaningful development history. The correct next step for provenance would be to obtain the original Git remote or repository URL from the maintainer and compare it against this archive hash-by-hash.

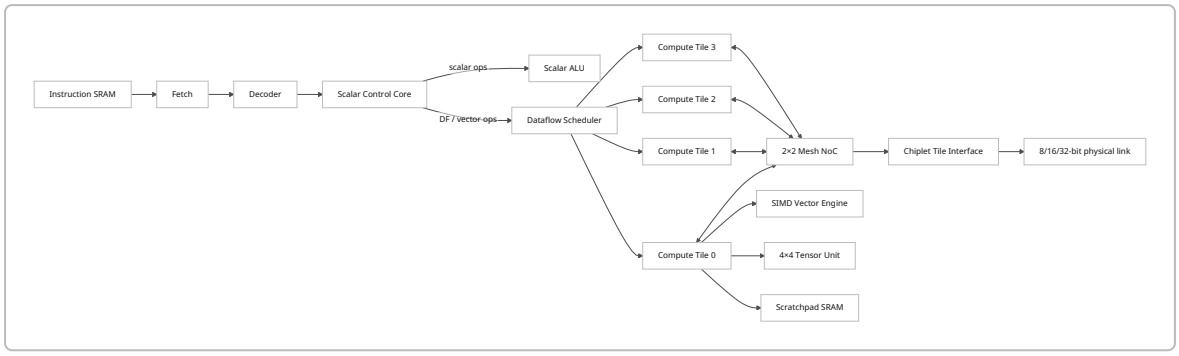
What the project is trying to build

The primary SilicaFlux core is a **hybrid scalar-plus-dataflow processor**. Scalar instructions move through a six-stage conceptual pipeline:

```
IF → ID → IS → EX → MEM → WB
```

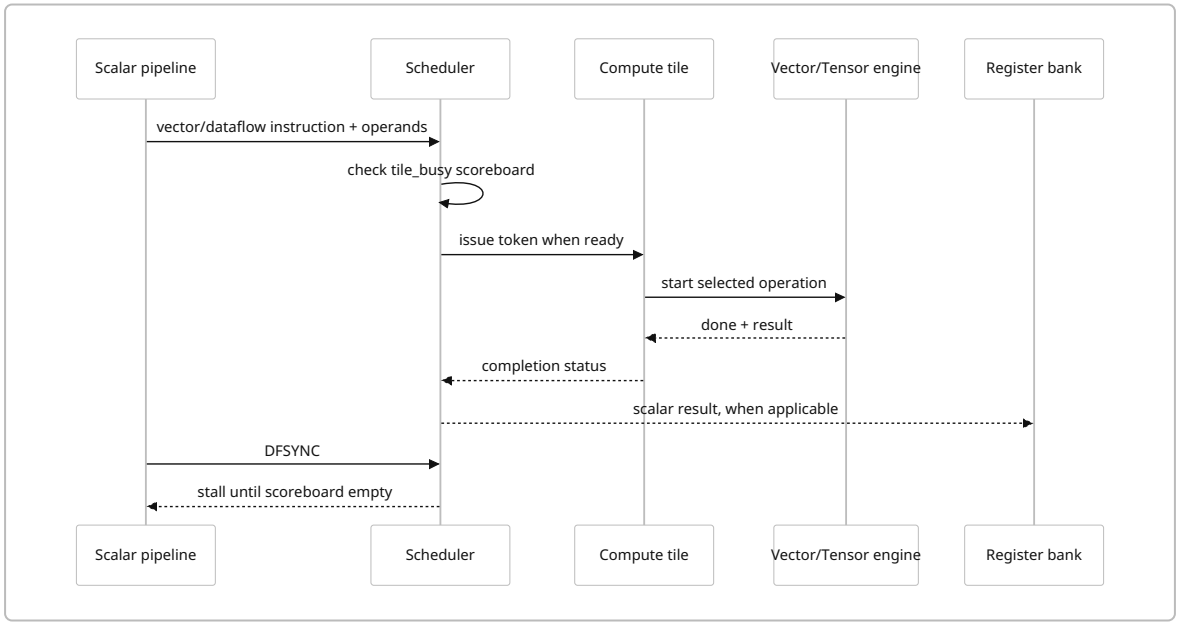
while vector/dataflow instructions can be dispatched to independent compute tiles through a scoreboard scheduler. A `DFSYNC` instruction acts as a barrier between asynchronous tile execution and the scalar pipeline.

The design has four important execution domains:



The ISA uses a 32-bit fixed instruction width with scalar R/I formats, a dataflow format carrying tile/address information, and a vector format. Shared architectural types are declared in `silicaflux_pkg.sv`, including decoded instructions, pipeline registers, work tokens, NoC flits and tile status structures. This is a sound HDL abstraction strategy because it makes interfaces explicit and centralises widths and opcodes.

The top-level dataflow is approximately:



The broader supplied collection is significantly larger than this canonical core. The earlier PDF audit reports 30,768 total lines across 179 RTL/test files, 176 defined modules, 284 assertions spread across 43 files and 43 testbench files. filecite turn0file0 Those figures describe the **whole portfolio**, not the cleaner 35-file `silicaflux/` subproject; conflating the two would overstate the verification maturity of the main architecture.

Repository/module map

Area	Principal components	Responsibility	Assessment
Shared architecture	`silicaflux_pkg.sv`	Widths, opcodes, decoded instruction structures, pipeline registers, work tokens, NoC flits, tile status	Strong centralisation; some nominal parameters are later hard-coded
Front end	`fetch_unit.sv`, `decoder.sv`	Instruction SRAM, PC sequencing, branch redirect, register-address extraction and decode	Clear separation of responsibilities
Scalar core	`control_core.sv`, `alu.sv`, `forwarding_unit.sv`	Scalar execution, branch handling, RAW forwarding/ interlocks and scheduler dispatch	Conventional in-order design; interfaces deserve stronger verification
Register state	`register_bank.sv`	32×32-bit scalar and 16×128-bit vector registers	Straightforward and reusable
Vector acceleration	`vector_engine.sv`	Lane-wise arithmetic, dot product, reduction and broadcast	Parameterised SIMD concept is good
Tensor acceleration	`tensor_unit.sv`	Sequential signed 4×4 integer matrix multiplication	Simple, deterministic, but high latency and fixed 4×4 geometry
Dataflow control	`scheduler.sv`	Tile scoreboard, dispatch, barrier handling and completion/write-back	Architecturally central; currently contains correctness risks
Tile integration	`compute_tile.sv`	Vector engine, tensor unit, local memory and token execution	Good architectural boundary
Memory	`memory_controller.sv`	Scalar, vector and burst scratchpad access	Useful accelerator-local memory abstraction
On-chip network	`noc_router.sv`, `noc_interconnect.sv`	XY routing across a 2×2 mesh	Deterministic and relatively easy to verify formally

Area	Principal components	Responsibility	Assessment
Chiplet link	`tile_interface.sv`	Flit serialisation/ deserialisation, credits and link status	Prototype quality; CDC/integrity treatment needs redesign
System integration	`silicaflux_top.sv`	Instantiates and wires the full cluster	Large integration point; 467 lines and a natural refactoring target
Tooling	`sfx_asm.py`, `run_sim.sh`	Assembly and simulation orchestration	Useful developer tooling, but no package/version locking
FPGA flow	Vivado/Quartus TCL, SDC/ XDC	Synthesis, timing and board constraints	Helpful prototype flow; measured QoR reports are absent
Documentation	`README.md`, `ISA_REFERENCE.md`	Architecture, ISA and usage	Better than average for prototype RTL; some documentation has drifted from implementation

The outer archive additionally includes stand-alone CPU, FSM, DSP, communications, formal-verification, cache and UVM-inspired projects, plus a substantially larger `SilicaFlux2.0` tree. These should be treated as **adjacent experiments or successor branches**, not as runtime dependencies of the primary core. Bundling them as ZIP files inside another repository makes code discovery, diffing and CI unnecessarily difficult.

Languages, interfaces, algorithms and dependencies

Languages and runtime/toolchain

The principal implementation language is **SystemVerilog**. The source uses modern constructs including `logic`, `package`, `typedef struct packed`, `enum`, `always_ff`, `always_comb`, `generate` and parameterised modules. Verilator explicitly supports these SystemVerilog constructs and provides a `--lint-only` mode suitable for automated static analysis. [2]

Python is used only for the assembler in the main core. Bash orchestrates Icarus Verilog simulation, and Tcl/SDC/XDC files provide vendor FPGA flows. The repository README requires Icarus Verilog 11.0+ and optionally GTKWave. The current Icarus documentation describes standard source and packaged installation, and its source files are distributed under GPL v2-or-later terms. [3] GTKWave identifies its project licence as GPL-2.0-or-later. [4]

AMD Vivado and Intel/Altera Quartus are optional synthesis tools rather than linked libraries. Both are separately licensed proprietary toolchains; AMD's current documentation describes tiered Vivado licences, while Intel's Quartus documentation directs users through its licensing infrastructure. [5]

Dependency and licence table

Dependency/ tool	Version in repository	Purpose	Licence / implication
Icarus Verilog	README says `11.0+`; comments mention compatibility work for v12	Compile and simulate SystemVerilog testbenches	GPL-2.0-or-later for the simulator itself; invoking the tool does not normally impose GPL on generated RTL artefacts. [6]
`vvp`	Bundled with Icarus version	Execute compiled simulation	Same Icarus project licensing context
Python	No exact version pinned	Run `sfx_asm.py`	Python distributed under the PSF licence; low redistribution friction. [7]
GTKWave	Unpinned, optional	View VCD waveforms	GPL-2.0-or-later. [8]
AMD Vivado	Unpinned	Xilinx/AMD FPGA synthesis and implementation	Proprietary/tiered licence; environment-dependent. [9]
Intel/Altera Quartus Prime	Unpinned	Cyclone V synthesis/ implementation	Proprietary licence terms and edition constraints. [10]
Verilator	Not in build, recommended addition	Lint/static analysis	External lint tool; current documentation supports automated `--lint-only` checking. [11]
SilicaFlux source itself	**No licence file found in supplied snapshot**	Project IP	**Critical ambiguity: no explicit permission to copy, modify, redistribute or commercialise should be assumed.**

The last row is the most important licence finding. A dependency's permissive or GPL licence does not solve the absence of an explicit licence for SilicaFlux itself.

The `SilicaFlux2.0` experimental web/ML projects introduce a separate dependency universe—React, Express, D3, Vite, Google GenAI and others—but they are not referenced by the main RTL build and should not be considered required dependencies of the processor. They should ideally be split into independent repositories with their own lockfiles, licences and security scanning.

Data model and public interfaces

SilicaFlux's "data models" are hardware data structures rather than database entities. The central abstractions are:

```
typedef struct packed {
    logic [4:0] opcode;
    logic [2:0] tile_id;
    logic [13:0] mem_addr;
    logic [4:0] length;
    logic [7:0] func;
    logic valid;
} df_token_t;
```

and a packed NoC flit carrying source/destination coordinates, payload, type and validity. This is an effective pattern: explicit packed records allow architectural information to move between modules without proliferating loosely related wires.

The most important module interfaces are:

Interface	Representative signature	Meaning
Cluster top	`silicaflux_top #(LANES=4)`	Clock/reset, instruction-memory programming port, external chiplet link, halt/debug outputs
Scalar ALU	`alu(a, b, op, result, flags...)`	Combinational scalar arithmetic
Vector engine	`issue_valid, opcode, func, vop_a, vop_b, scalar_in → result_valid, vresult, scalar_result`	SIMD execution API
Tensor unit	`start + 4 A rows + 4 B columns → busy/done + 4 result rows`	Fixed 4×4 matrix-multiply interface
Scheduler	decoded instruction + scalar/vector operands + tile ready/status → token fields + write-back + `sync_stall`	Dataflow dispatch/control plane
Memory controller	request opcode/address/data/length → ready/response/alignment status	Scratchpad transaction interface
NoC router	local/north/south/east/west ready-valid ports	Mesh routing
Tile interface	NoC flit ready-valid ↔ narrow physical credit link	Chiplet boundary

There are no HTTP endpoints or conventional software APIs in the principal design. The public interfaces are RTL ports, ISA encodings and the assembler command line.

Key algorithms

The scalar pipeline uses familiar in-order forwarding and interlock logic. Load-use hazards generate a one-cycle stall, while branch resolution redirects fetch. The architecture deliberately avoids speculation and out-of-order execution, which simplifies verification and produces predictable timing.

The dataflow scheduler maintains a four-bit busy scoreboard. An operation can be dispatched when the selected tile is idle and ready; completion clears that tile's busy bit. `DFSYNC` activates a barrier until the scoreboard becomes empty. Conceptually:

```
if (token_accepted[t])
    tile_busy[t] <= 1'b1;

if (tile_done[t])
    tile_busy[t] <= 1'b0;

sync_stall = barrier_active && (tile_busy != 0);
```

The vector engine implements lane-parallel arithmetic and collective operations. This is the likely throughput-critical part for inference/DSP workloads because one instruction amortises control cost over multiple lanes.

The tensor unit implements a **sequential 4×4×4 matrix multiplication**: 16 output elements × four multiply-accumulate steps = 64 MAC cycles, plus load/finish states. The source comment therefore states approximately 66 cycles. This directly contradicts the README's earlier "~20 cycle" claim and should be corrected. More importantly, the design has one active MAC datapath rather than a 16-MAC systolic array, so "tensor unit" here denotes a functional accelerator, not a high-throughput matrix engine comparable with modern ML accelerators.

XY routing in the NoC is deterministic: route in one dimension and then the other. This eliminates adaptive-routing state and makes deadlock reasoning easier for a simple mesh, but also means congestion cannot be routed around.

Verification, build, CI/CD and deployment

The primary core has a convenient local test launcher:

```
iverilog -g2012 ... rtl/*.sv tb/<test>.sv
vvp <compiled-test>
```

and seven named targets: vector, tensor, memory, scheduler, NoC, tile and full top-level. The script uses strict Bash settings (`set -euo pipefail`) and emits separate compile and run logs. Icarus is a reasonable open-source baseline simulator, while adding Verilator would materially improve lint coverage; its current documentation recommends lint-oriented warning checking and supports `--lint-only`. [12]

The main weakness is the **oracle**, not the number of testbenches. The wider PDF audit already noted that the project relies heavily on straightforward `initial` / `$display`-style simulation rather than complete coverage-driven verification. filecite turn0file0 Inspection of the canonical core makes that concern sharper: only some tests compare outputs against expected values; several integration tests merely run a scenario and then increment a pass counter or ask the engineer to inspect waveforms. `tb_silicaflux_top.sv`, for example, contains a reusable checking task but several top-level tests do not use it to validate the architectural registers they claim to exercise.

The simulation runner determines success by counting log lines beginning with `[FAIL]`. Therefore a test that never performs a check can still be reported as passing. The script even labels the number of `[PASS]` messages as "assertions passed", despite the canonical core having essentially no SVA-based assertion suite.

Test and CI status

Area	Present?	Current mechanism	Assessment
Unit simulation	Yes	Seven SV testbenches under `tb/`	Useful smoke coverage
Full-system simulation	Yes	`tb_silicaflux_top.sv`	Insufficiently self-checking
Assertions/SVA	Negligible in canonical core	Almost none in RTL/TB	Major gap
Functional coverage	No evidence	None found	Major verification gap
Constrained random	No	Directed stimulus	Adequate for bring-up, not closure
Scoreboards/ reference models	Very limited	Direct procedural comparisons	Needs architectural reference model
Formal verification	Not wired into main core	Separate archive exists elsewhere in repository	Valuable work exists but is disconnected
Lint	No repository automation	Prior PDF mentions prototype/ static lint work	Add Verilator/Verible as gating CI
GitHub Actions	Not found	—	No hosted CI
GitLab CI	Not found	—	No hosted CI
Bitbucket Pipelines	Not found	—	No hosted CI
FPGA synthesis	Yes	Vivado and Quartus Tcl	Good prototype support
Timing constraints	Yes	`silicaflux.sdc` + board XDC	Useful starting point
Bitstream deployment	Scripted	Vendor-generated `.bit` / `.sof`	Board-specific, not release-managed
ASIC flow	No production flow	Conceptual/documentary only	Not tapeout-ready

The broader PDF reports 284 assertions across the entire multi-project collection, which is materially better than the canonical core alone, but it also concludes that formal/UVM-style verification is limited and that basic testbenches dominate. filecite turn0file0 The correct interpretation is therefore: **verification expertise is represented somewhere in the repository, but verification is not systematically applied to the main SilicaFlux design**.

The synthesis flow targets 100 MHz through a shared 10 ns SDC constraint. Vivado is configured for an Arty A7-100T and Quartus for a DE10-Nano/Cyclone V. The README estimates 8–12k LUTs, 6–9k FFs, eight 36K BRAM-equivalents, 16–24 DSP blocks and 80–110 MHz, but no generated utilisation/timing report is stored alongside the canonical project. These figures must therefore be treated as **design estimates, not measured benchmark results**.

There is no evidence of packaging, release automation, signed artefacts, reproducible tool versions, Docker/Nix environments or generated timing/coverage dashboards. In comparison, mature open hardware projects commonly run lint automatically; lowRISC explicitly describes running Verilator lint across SystemVerilog sources. [13]

Security, correctness and performance analysis

Security and correctness risks

Hardware security should be assessed in terms broader than software CVEs. MITRE maintains a dedicated Hardware Design CWE view spanning general circuit logic, memory, interconnects, privilege separation, debug/test, clock/reset and interface concerns. [14] SilicaFlux currently has no security architecture describing trust boundaries, privilege levels, memory protection, secure boot, authenticated debug or chiplet-link authentication.

The principal findings are:

Severity	Finding	Consequence
High	Multi-bit `phy_rx_data` is synchronised by independently registering the full bus twice	If the physical link is actually asynchronous to `clk`, individual bits can settle differently; a coherent word is not guaranteed
High	Scheduler completion write-back does not preserve an explicit destination register and assigns `wb_reg <= '0`	Tile scalar results cannot reliably return to their intended architectural register
High	Scheduler ignores scalar result value `0` (`result != '0` condition)	A legitimate zero result may never be written back
High	Project has no explicit source licence	Legal/commercial reuse risk rather than technical exploit
Medium–High	Instruction-memory programming interface is exposed at the cluster top with no access-control concept	In a deployed device, writable execution memory would require a trust model; MITRE specifically catalogues inadequate restriction of software-visible hardware features as a hardware weakness class. [15]
Medium	"Parity" is calculated over assembled data but no transmitted parity bit is present	It cannot detect arbitrary link corruption as currently described

Severity	Finding	Consequence
Medium	Reset and physical-link assumptions are weakly specified	CDC/reset release issues can become silicon-only faults
Medium	No assertion suite for ready/valid, FIFO-like ownership, NoC routing or scheduler invariants	Protocol violations may survive directed simulation
Medium	`NUM_TILES` is parameterised but scheduler logic enumerates tile 0–3 explicitly	Configuration can compile incorrectly or behave unexpectedly if scaling is attempted
Medium	Debug/programming features are not isolated by build configuration	Production hardware commonly needs explicit restriction/removal of debug paths; OWASP likewise warns about leaving hardware debug interfaces enabled. [16]
Low–Medium	No ECC in memories, no fault reporting strategy beyond limited status bits	Reduced robustness for ASIC/chiplet use

The CDC issue deserves special emphasis. A 2-FF synchroniser is appropriate for a **single control bit** whose receiving domain can tolerate synchronisation latency. It is not generally sufficient to pass a multi-bit word by independently registering all bits. A robust chiplet link should use a source-synchronous clock/strobe, asynchronous FIFO, handshake-stabilised holding register, Gray-coded pointer protocol or a proper PHY/protocol layer.

Likewise, the memory and execution interfaces have no notion of privilege or protection domains. That is acceptable for a laboratory accelerator but incompatible with claims of a secure general-purpose SoC without additional architectural controls. MITRE's hardware weakness taxonomy specifically includes interface/IO, memory and access-control classes. [17]

No meaningful "known CVE" result applies to the main RTL because it does not consume a traditional package ecosystem. The highest-risk vulnerabilities are **design-level logic and integration weaknesses**, not dependency CVEs.

Performance characteristics

The scalar engine is deliberately simple: in-order execution, forwarding and a one-cycle load-use bubble. Its strength is determinism and low control complexity, not scalar IPC.

The SIMD engine provides the most straightforward performance scaling. With four 32-bit lanes in a 128-bit vector register, one VADD/VMUL logically performs four lane operations. Increasing `LANES` can improve throughput, but only if the register/vector width, memory bandwidth and tile integration scale consistently; the repository documentation implies wider lane configurations more freely than several fixed 128-bit interfaces actually permit.

The tensor unit is currently **latency-oriented, not throughput-oriented**. A 4×4 result requires 64 serial multiply-accumulate steps, so the implementation is closer to a compact matrix functional unit than an AI tensor array. A first major performance refactor would be to use 4, 8 or 16 parallel MACs, pipeline accumulation and optionally support INT8/INT16 packing.

The 2×2 mesh itself is small enough that deterministic XY routing is sensible. At larger dimensions, however, a single-flit/limited-buffer implementation and fixed XY routing can produce hot spots. There is no traffic generator, saturation-throughput test, latency histogram or back-pressure stress suite, so NoC performance has not been quantitatively established.

The chiplet interface serialises a 43-bit logical flit over a default 8-bit physical link, requiring six beats after width rounding. Ignoring protocol bubbles, that alone places a hard upper bound of roughly one flit per six link cycles in each direction. Credits, synchronisation and state-machine overhead reduce effective throughput further. A real chiplet design would normally separate logical NoC width from an explicitly specified PHY encoding, framing, CRC/ECC, training and CDC mechanism.

The README's claimed 80–110 MHz FPGA Fmax and resource ranges should be considered preliminary because no associated post-route timing reports are present. The correct performance programme is to record actual synthesis tool versions, utilisation, worst negative slack, achieved Fmax, power estimate, benchmark cycle counts and traffic-level throughput under CI.

Developer experience, maintainability and recommended refactoring

Developer experience is mixed. At the **core-project level**, the README is unusually useful: architecture diagrams, ISA formats, simulation commands, assembler usage and FPGA synthesis commands are all present. The assembler includes example programs for Fibonacci, parallel dot product, 4×4 matrix multiplication and approximate softmax. Module naming is generally descriptive, and the code is easier to navigate than the broader PDF's aggregate metrics might suggest.

At the **repository level**, however, packaging works against contributors. Core source trees are embedded in ZIP files, related projects are duplicated at multiple versions, `.sf` and `.sv` variants coexist, and the outer README describes a neat `rtl/`, `tb/`, `sim/`, `synth/` layout that does not exist at the outermost level without first opening `silicaflux.zip`. This prevents ordinary Git diffs, code review, blame, CI path filters and dependency scanning from operating effectively.

The supplied PDF reached a similar broad conclusion: parameterisation and modularity were good, while coding consistency and verification sophistication lagged mature industrial RTL. 【filecite】turn0file0【】 A rigorous improvement sequence would be:

1. **Normalise the repository first.** Extract the canonical core directly into version control; move experimental projects into `examples/`, `research/` or separate repositories; delete duplicate archives; publish an explicit `LICENSE`, `CONTRIBUTING.md`, architecture version and changelog.

2. **Fix correctness before optimisation.** Preserve the destination register in scheduler tokens/completion state; permit zero-valued write-back; make tile arrays genuinely parameterised; reconcile tensor latency and all other documentation against RTL.

3. **Redesign the chiplet boundary.** Replace the unsafe multi-bit 2-FF crossing with a defined CDC/PHY protocol. Add framing and CRC/ECC, link-reset/training semantics, replay or error policy and explicit clock-domain assumptions.

4. **Create executable specifications.** Implement a lightweight Python/Scala/C++ ISA reference model and compare architectural state after every instruction/program. Turn every integration test into a self-checking test with non-zero exit status on mismatch.

5. **Add assertions and formal proofs.** Priority candidates are `valid -> stable until ready`, no double-issue to busy tiles, `DFSYNC` completion, NoC destination correctness, one-hot grant invariants, register-zero invariants, memory alignment and tile completion liveness. The repository already contains separate formal-verification material; the useful next step is to bring properties into the primary design rather than keeping them as an isolated exercise. The broader PDF also recommends integrating formal methods. filecite turn0file0

6. **Build real CI.** At minimum: Verilator lint, Icarus simulation, assembler unit tests, formal smoke checks and one synthesis elaboration target. Verilator provides a dedicated lint-only flow and detailed warning classes suitable for making lint a required gate. [12] Pin tool versions in a container.

7. **Introduce coverage.** Record line/toggle/FSM coverage where the simulator permits it, and functional coverage for all ISA opcodes, hazards, routing paths, burst lengths and synchronisation cases. The existing broader audit explicitly identified the lack of coverage-driven verification as a weakness. filecite turn0file0

8. **Refactor performance-critical blocks after measurement.** Pipeline or parallelise the tensor MAC, benchmark SIMD configurations, add NoC traffic tests and evaluate memory-bank conflicts. Do not optimise based solely on the README's unverified resource/Fmax estimates.

Priority refactoring backlog

Priority	Change	Why it matters
P0	Correct scheduler result/destination handling	Architectural correctness
P0	Replace chiplet multi-bit CDC	Silicon reliability/data integrity
P0	Add project licence	Enables legitimate collaboration/reuse
P0	Extract ZIP-contained source into normal Git tree	Makes all other engineering workflows practical
P1	Make testbenches genuinely self-checking	Prevents false-green regression runs
P1	Add SVA + formal checks	Catches control/protocol bugs directed tests miss
P1	Add CI with lint/sim/formal	Establishes repeatability
P1	Remove hard-coded four-tile logic	Restores claimed configurability
P2	Pipeline/parallelise tensor unit	Large performance opportunity
P2	Add benchmark and synthesis report retention	Converts estimates into evidence

Priority	Change	Why it matters
P2	Split top-level integration into interfaces/ subsystems	Reduces coupling and review complexity
P2	Version the ISA and assembler together	Prevents RTL/tooling drift

The maintenance risk is currently **moderate-to-high**. The architecture is understandable and modular enough to improve, but claims such as "semiconductor-grade", "chiplet-ready" and FPGA performance estimates run ahead of the verification evidence. The supplied broader audit's approximate 65/100 quality assessment is directionally reasonable for the portfolio, but for the canonical SilicaFlux core I would describe the state more precisely as **well-documented advanced prototype RTL, not production-qualified IP**. filecite turn0file0

Overall assessment and decision points

SilicaFlux has a coherent architectural idea: keep scalar control simple, offload parallel work to independently scheduled SIMD/tensor tiles, move data through local scratchpads and a deterministic mesh, and expose the same fabric through a chiplet boundary. The project demonstrates solid understanding of processor pipelines, custom ISAs, ready/valid hardware protocols, vector arithmetic, memory systems and FPGA tooling.

Its principal engineering gap is not conceptual architecture; it is **evidence of correctness**. The next stage should therefore not be adding more accelerators or more feature branches. It should be narrowing the repository to a canonical implementation, fixing known control/CDC problems, making all tests self-checking, adding formal properties and CI, and collecting reproducible synthesis/ performance measurements.

The public-hosting investigation does not presently establish an externally maintained "main" repository, stars or recent commit history. The exact queries used were:

```
"SilicaFlux" GitHub
"Silica Flux" SystemVerilog GitHub
site:gitlab.com SilicaFlux SystemVerilog
site:bitbucket.org SilicaFlux
"A Modular Dataflow-Oriented Semiconductor Compute Architecture"
"SilicaFlux v1.0" semiconductor architecture RTL
"SilicaFlux" "Dataflow Scheduler" "DFSYNC"
"silicaflux_top" "tensor_unit" SystemVerilog
site:github.com "SilicaFlux" "README.md"
site:github.com "SilicaFlux" "silicaflux_pkg.sv"
site:github.com "SilicaFlux2.0"
site:github.com "silicaflux-bitcoin-asic"
```

No indexed result matched the supplied implementation; superficially similar hits were unrelated projects such as SiliconFlow and ByteDance Flux. [1] Until an origin URL is supplied, the archive should therefore be treated as an **unversioned/private snapshot**. The most useful provenance next step is to obtain the original Git remote, preserve commit history and tags, add an explicit licence, and compare that source against the supplied archive before making any commercial, security or maintenance judgement dependent on provenance.

1 SiliconFlow · GitHub
https://github.com/siliconflow?utm_source=chatgpt.com

2 11 Verilating — Verilator 5.054 documentation
https://verilator.org/guide/latest/verilating.html?utm_source=chatgpt.com

3 Installation Guide - Icarus Verilog documentation
https://steveicarus.github.io/iverilog/usage/installation.html?utm_source=chatgpt.com

4 GitHub - gtkwave/gtkwave: GTKWave is a fully featured GTK+ based wave viewer for Unix and Win32 which reads LXT, LXT2, VZT, FST, and GHW files as well as standard Verilog VCD/EVCD files and allows their viewing. · GitHub
https://github.com/gtkwave/gtkwave?utm_source=chatgpt.com

5 9 AMD Vivado™ Licensing Options | Flexible Subscription & Perpetual Tiers
https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado/vivado-licensing-options.html?utm_source=chatgpt.com

6 iverilog/main.cc at master · steveicarus/iverilog · GitHub
https://github.com/steveicarus/iverilog/blob/master/main.cc?utm_source=chatgpt.com

7 History and License — Python 3.13.18 documentation
https://docs.python.org/3.13/license.html?utm_source=chatgpt.com

8 gtkwave/meson.build at master · gtkwave/gtkwave · GitHub
https://github.com/gtkwave/gtkwave/blob/master/meson.build?utm_source=chatgpt.com

10 Intel® FPGA Software Download and Installation Support
https://www.intel.com/content/www/us/en/support/programmable/licensing/installation-and-licensing.html?utm_source=chatgpt.com

12 Errors and Warnings — Verilator 5.054 documentation
https://verilator.org/guide/latest/warnings.html?utm_source=chatgpt.com

13 Ibex: Code with Confidence - lowRISC
https://lowrisc.org/news/ibex-code-with-confidence/?utm_source=chatgpt.com

14 CWE - CWE-1194: Hardware Design (5.0)
https://cwe.mitre.org/data/definitions/1194.html?utm_source=chatgpt.com

15 CWE - CWE-1256: Improper Restriction of Software Interfaces to Hardware Features (4.20)
https://cwe.mitre.org/data/definitions/1256.html?utm_source=chatgpt.com

16 7. Broken Zones and Conduits Design - OWASP Operational Technology (OT) Top 10
https://ot.owasp.org/v/2025/the-top-10/broken-zone-and-conduits-design/?utm_source=chatgpt.com

17 CWE - 2025 CWE Most Important Hardware Weaknesses
https://cwe.mitre.org/topHW/archive/2025/2025_MIHW_key_insights.html?utm_source=chatgpt.com